

# Queue ADT, Operations & Array Implementation

**Q.** Define Queue as an Abstract Data Type (ADT). Explain its standard operations and demonstrate its implementation using an Array with a relevant code snippet.

---

> **Definition to Remember**

---

> A **Queue** is a linear data structure in which the elements are inserted at one end and removed from the other end. It follows the First In First Out (FIFO) principle.

## 1. Queue as an ADT

As an Abstract Data Type, a Queue manages a collection of elements and restricts access to two specific ends:

---

- **Enqueue**: Add an element to the rear of the queue.

## 2. Standard Operations

| Operation | Description                                    | Condition           |
|-----------|------------------------------------------------|---------------------|
| Enqueue   | Add an element to the rear of the queue.       | Queue is not full.  |
| Dequeue   | Remove an element from the front of the queue. | Queue is not empty. |

## 3. Array Implementation in C

Two pointers (`front` and `rear`) are initialized to `-1`.

```
```c
#include <stdio.h>
#define SIZE 100

int queue[SIZE];
int front = -1;
int rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Queue is full\n");
        return;
    }
    rear++;
    queue[rear] = value;
    printf("Element %d is enqueued\n", value);
}

int dequeue() {
    if (front == -1) {
        printf("Queue is empty\n");
        return -1;
    }
    int value = queue[front];
    front++;
    printf("Element %d is dequeued\n", value);
    return value;
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    dequeue();
    dequeue();
    return 0;
}
```
```

#### 4. Advantages and Limitations

- **Advantage:** Array implementation is simple and fast ( $O(1)$  time complexity for enqueue/dequeue).

---

> **Must-Write Points (for 10 marks)**

---

> **Quick Recall**

---

#### Complete Executable C Code: Linear Queue Array Implementation

```
```c
```

```
#define MAX 5
```

```
struct Queue {
```

```
void initQueue(struct Queue *q) {
```

```
int isFull(struct Queue *q) {
```

```
int isEmpty(struct Queue *q) {
```

```
return
```

```
void enqueue(struct Queue *q, int val) {
```

```
if (isFu
```

```
int dequeue(struct Queue *q) {
```

```
if (isEr
```

```
void display(struct Queue *q) {
```

```
if (isEr
```

```
int main() {
```

```
struct
```

```
enqueue(&q, 10);
```

```
enque
```

```
printf("Dequeued: %d\n", dequeue(&q));
```

```
display
```

```
return 0;
```

```
}
```